ngspice Linear Solvers — KLU vs. Sparse 1.3

Behavior, defaults, and limitations across DC / AC / transient / noise

Ngspice + OpenVAF Enhancements project

July 2026

Contents

ngspice linear solvers — KLU vs. Sparse 1.3 (behavior, defaults, limitations)

This build of ngspice-46 ships two direct linear solvers, and they are not fully interchangeable across analysis types. This note records exactly how they differ — which is the default, how to select and confirm each, and where KLU falls short — based on a direct read of the source and a solver-by-solver sweep of the whole `examples/` suite.

TL;DR

	KLU	Sparse 1.3
Availability in this build	Compiled in (SuiteSparse, statically linked — 44 `klu_*` symbols in the binary)	Always present (ngspice's own solver)
Default?	No	Yes — this build defaults to Sparse 1.3
How to select	`.option klu`	`.option sparse` (or just the default)
DC op / DC sweep	✓ correct	✓ correct
AC	✓ correct	✓ correct
Transient	✓ correct (one caveat below)	✓ correct
Noise (**`.noise`**)	✓ correct (since E-113)	✓ correct
Pole-zero (**`.pz`**)	✓ single-ended; ⚠balanced-output Sparse-only	✓ correct
Sensitivity (**`.sens`**, DC & AC)	✓ correct (since E-114)	✓ correct
Distortion (**`.disto`**)	✓ correct (since E-115)	✓ correct
Periodic steady state (**`.pss`**)	✓ correct (since E-118)	✓ correct
Periodic small-signal (**`.pac`**/**`.pnoise`**/**`.pxf`**/**`.psp`**)	✓ correct	✓ correct
Transient checkpoint/restart (**`savestate`**/**`loadstate`**)	⚠Sparse-only (E-131)	✓ correct

The periodic small-signal analyses (PAC / Pnoise / PXF and the E-132 periodic S-parameters `.psp`) build a dense `(2M+1)N` harmonic conversion matrix and solve it with a standalone dense complex LU (`pss_csolve`) that is independent of the sparse linear solver — so they inherit the solver only through the underlying PSS, which runs correctly under both since E-118 (KLU re-factors every shooting step, so PSS is *slower* under KLU but not wrong). The E-133 two-tone `qpss` and the E-134 frequency-domain `hb` (harmonic balance) are likewise solver-independent — `qpss` drives a transient and direct-DFTs it; `hb` does a dense complex Newton on the conversion matrix and uses

the sparse solver only to *read* the periodic `G(t)/C(t)` off the device matrix. That read is the one solver-specific detail: Sparse uses `spSetComplex`, KLU needs the complex-CSC binding (`DEVbindCSCComplex`), exactly as the PAC harmonic extraction does — `hb` carries the same `#ifdef KLU` branch, so it is verified bit-identical under **`.option klu`** and **`.option sparse`** (a bare `hb` also copies the task's KLU mode before building the matrix, so `.option klu` takes effect without a prior analysis). Transient checkpoint/restart (Enhancement-131) is the one feature that is genuinely Sparse-only: on the restore path KLU's symbolic/numeric factorization objects are absent (they are only built during a full run's operating point), so `savestate`/`loadstate` reject `.option klu` with a clear message rather than crash.

Practical guidance: leave the default (Sparse 1.3) unless you have a specific reason to switch. Sparse 1.3 runs every analysis in the suite. Since Enhancement-113 KLU also runs noise and single-ended pole-zero correctly, and since Enhancement-114 it runs DC/AC sensitivity, and since Enhancement-115 distortion (**`.disto`**) correctly; the only *analysis* still Sparse-only under KLU is balanced-output pole-zero (the transient checkpoint/restart *feature* is also Sparse-only, per the table above). Reach for `.option klu` on large, sparse DC/AC problems where KLU's ordering and factorization are faster, and — because its symbolic ordering is computed once and cannot re-pivot dynamically like Sparse — expect it to be less forgiving of a near-singular Jacobian on a stiff transient edge, which the dissipative Gear integrator avoids (see the former opamp741 discrepancy below).

Tuning KLU (Enhancement-152): KLU's reordering and scaling, previously hard-coded, are now `.options` — `klu_ordering=amd|colamd` (fill-reducing ordering), `klu_scale=none|sum|max` (row equilibration), and `klu_btf=on|off` (block-triangular-form permutation), with defaults `amd`/`max`/`on`. They change only *how* the matrix factors, never the solution (verified physically identical to ~1e-14 across all settings), and matter only on unusual matrix structures (a different ordering can reduce fill) or badly-scaled matrices. The `klu_memgrow_factor` knob was also fixed (it had silently collapsed to a boolean).

Selecting and confirming the solver

The netlist option is a simple flag:

```
.option klu       * use KLU (SuiteSparse)
.option sparse    * use the legacy Sparse 1.3  (this is also the default)
```

Because unknown `.option` keywords are silently ignored by ngspice, ”it ran without complaint” is *not* evidence the solver changed. To confirm which solver is actually active, use the interactive `option` command, which prints the live `CKTkluMODE`:

```
.control
run
option        * prints, among other things:  "Matrix solver:  KLU"  or  "Sparse 1.3"
.endc
```

On the committed `bin/` binary this reports **`Sparse 1.3`** for a bare deck and for `.option sparse`, and **`KLU`** only when `.option klu` is given — confirming Sparse 1.3 is the default here.

Noise and pole-zero under KLU (Enhancement-113)

ngspice used to refuse both outright (`noisean.c` / `pzan.c` printed ”not (yet) supported with 'option KLU'”). The real reason for noise was subtler than ”not wired up”: noise uses the adjoint method — it solves the *transposed* system $A^T \cdot x = e$ via `SMPcaSolve` — and `SMPcaSolve`'s KLU branch was calling the non-transposed `klu_z_solve` where its Sparse branch calls `spSolveTransposed`. That silently produced the wrong noise for any asymmetric matrix (every circuit with a transistor or controlled source), which is why it was disabled rather than merely slow.

Enhancement-113 fixes the adjoint solve (`klu_z_tsolve`) and lifts the guards, so under KLU:

- Noise is correct — verified identical to Sparse on resistor thermal noise, asymmetric VCCS circuits, OSDI device models, and integrated totals. (`.sp` S-parameters share the same adjoint solve and are corrected too.)
- Single-ended pole-zero (grounded output reference) runs correctly.
- Balanced-output pole-zero (non-grounded reference) stays Sparse-only: its zeros phase folds columns at solve time, which Sparse survives via dynamic Markowitz re-ordering but KLU's fixed symbolic factorization cannot; a targeted guard now directs that case to `.option sparse`.

Enhancement-114 then fixes sensitivity under KLU. Sensitivity builds an auxiliary perturbation matrix `delta_Y` that is a plain Sparse matrix (it is only multiplied, never factored); two KLU setup blocks were gated on the *main* matrix's flag and dereferenced the NULL `delta_Y->SMPkluMatrix`, segfaulting on every DC/AC `.sens` deck. Gating them on `delta_Y`'s own flag keeps it Sparse under KLU too, so both DC and AC sensitivity now match Sparse exactly.

- Distortion (`.disto`) was Sparse-only — `distoan.c` had no KLU code, so under `.option klu` the (complex) distortion solves ran against a matrix left in real mode and silently produced no output. Surfaced while validating E-114. Enhancement-115 fixes it: it converts the KLU matrix real↔complex around the distortion solve loop (exactly as `acan.c` does for AC), so `.disto` now matches Sparse bit-for-bit.

KLU discrepancies — all resolved

Beyond balanced-output pole-zero (a genuine "not (yet) supported" *skip*, not a wrong result), no example now produces a different result under KLU than under Sparse. `KLU_XFAIL` is empty; the harness runs every example under both solvers and expects agreement.

opamp741 — was the last one; the real cause was the integration method, not KLU. The transistor-level opamp741 example (a μA741 from ~70 lines of Verilog-A BJT) used to abort under KLU on its large-signal slew test (`pulse(-5 5 …)` into the follower): output-stage transistors switch off at the slewing edge, their transconductances collapse, KLU declares the Jacobian singular (`x1.o1/o2/b34/cm`), the timestep collapses, and ngspice aborted at t $\approx$ 2.03 μs — while Sparse ran the full `tran 20n 80u` cleanly.

The near-singular Jacobian at the edge was manufactured by the default trapezoidal integrator, which is non-dissipative and *rings* on the stiff feedback slew, driving the output transistors hard off. It is that ringing — not KLU's linear solve — that produced the collapse: Sparse's dynamic Markowitz re-ordering happened to survive the near-singular step where KLU's fixed symbolic ordering could not, but the step should never have been that singular. Switching the two transient decks to Gear (`.option method=gear`, dissipative BDF-2) damps the ringing so the edge is never near-singular: the slew now runs to completion under both solvers and the results agree to ~8 sig figs (final V(out) −3.3153833712 KLU vs −3.3153833649 Sparse), with every figure of merit (Aol, fu, PM, slew rate, offset, swing) identical between the solvers. opamp741 was removed from `KLU_XFAIL` accordingly.

(This corrects the earlier reading of this case as an unfixable KLU linear-solver limitation needing a hybrid Sparse-fallback: the earlier pivoting/ordering knobs [E-116] left the *trapezoidal* run's abort unchanged because they addressed the symptom, not the ringing that caused it. The genuinely structural KLU issues were the two below, which E-116 did fix.)

Two former discrepancies, now fixed (E-116). `groundcontrib` (node-to-ground `V(p,gnd) <+ 1.5` read `v(p)=0` under KLU instead of `1.5`) and `hierbranch` (hierarchical branch-*current* probes read `0`) were both structural, not numerical: an OSDI internal node that appears in no Jacobian entry — a `ground` reference whose branch contribution drops its column — was allocated its own all-zero solver row, which Sparse tolerates but KLU treats as structurally singular. Enhancement-116 ties such decoupled internal nodes to ground at setup, so both now match Sparse under KLU.

The dual-solver test harness

Every `verify_*.py` in `examples/` is run under both solvers automatically. Each script calls `check_both_solvers(__file__)` (right after importing `_setup`); on a normal run that re-executes the script once per solver — injecting `.option sparse` / `.option klu` into every ngspice deck — and prints a combined verdict:

```
=== BOTH-SOLVER RESULT [ceil]: sparse=PASS  klu=PASS => OK ===
```

KLU's one remaining unsupported analysis (balanced-output pole-zero) is auto-detected and reported `SKIP`; `KLU_XFAIL` is now empty (opamp741, its last member, was resolved — see above). A separate `SPARSE_ONLY` registry (`{rfanalyses, rfpss}`) marks the heavy periodic-steady-state examples whose KLU pass is not *wrong*, just slow — KLU re-factors every PSS shooting step, so a 1024-sample `.pss` that is a couple of minutes under Sparse becomes 10–15 min under KLU. Those run Sparse-only by default (reported `klu=SKIP`); their KLU correctness is a property of the ngspice source, verified once when E-118 landed, not something worth re-paying every sweep. Env escape hatches: `NGSPICE_SOLVER=klu|sparse` runs once under one solver; `NG_BOTH=0` disables the dual run; **`NG_SLOW_KLU=1`** forces the skipped heavy-PSS KLU pass back on (to re-check the E-118 KLU-PSS fix on demand).

Sweep result (all 101 machine-checkable scripts): 101/101 OK — every example is `sparse=PASS` with `klu` in `{PASS, SKIP, XFAIL}`, i.e. the two solvers agree wherever KLU is applicable. (The `linesearch` example initially *crashed* under KLU, which Enhancement-112 fixed; Enhancement-113 then moved 10 examples from KLU-skipped to KLU-passing by enabling noise and single-ended pole-zero; Enhancement-114 fixed the AC/DC-sensitivity crash under KLU, and — by fixing the deck-injector restore bug — un-masked two more `XFAIL`s (`groundcontrib`, `hierbranch`); Enhancement-115 then fixed distortion, so the `analyses` example now runs fully under KLU; and Enhancement-116 fixed `groundcontrib` and `hierbranch` at their shared root cause; and `opamp741` — the last `XFAIL` — was resolved by running its stiff transient under Gear rather than the default trapezoidal method, so **`KLU_XFAIL`** is now empty.)

Conclusion: for DC, AC, transient, noise, single-ended pole-zero, sensitivity, and distortion the two solvers agree across the entire suite (`KLU_XFAIL` is empty), while balanced-output pole-zero is Sparse-1.3-only by ngspice design. Sparse 1.3, the default, covers everything — which is why it is the default and why the dual-solver harness keys correctness off it.